

Introduction to Computing (CS 101)

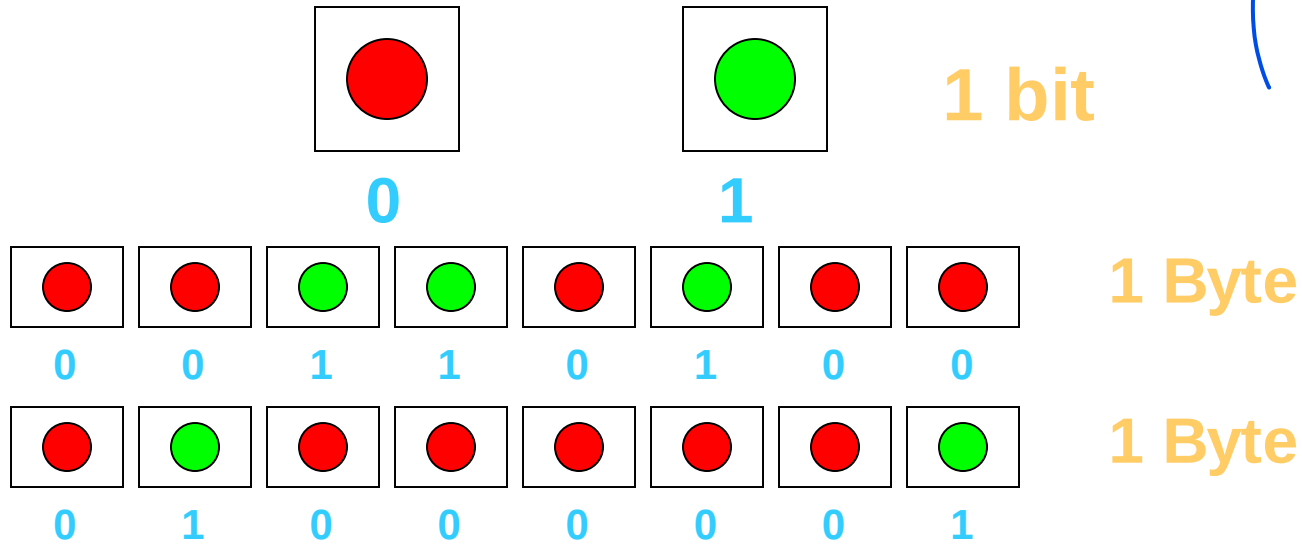
Hexadecimal Notation and Floating Point Number Representation

T. Venkatesh & John Jose



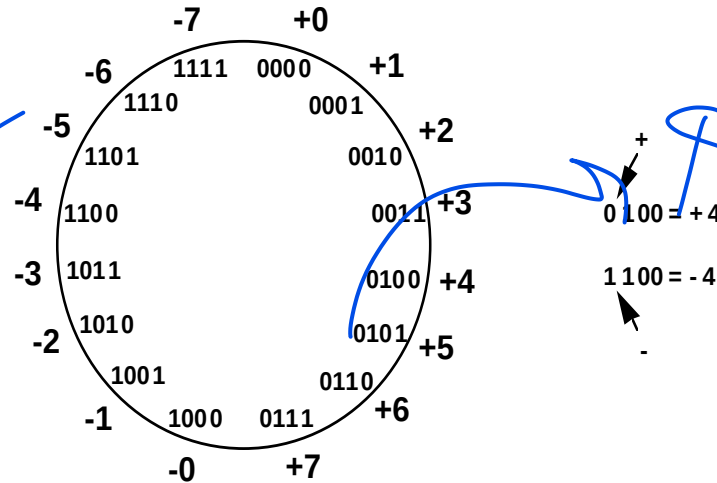
**Department of Computer Science & Engineering
Indian Institute of Technology Guwahati**

Review of Binary Storage

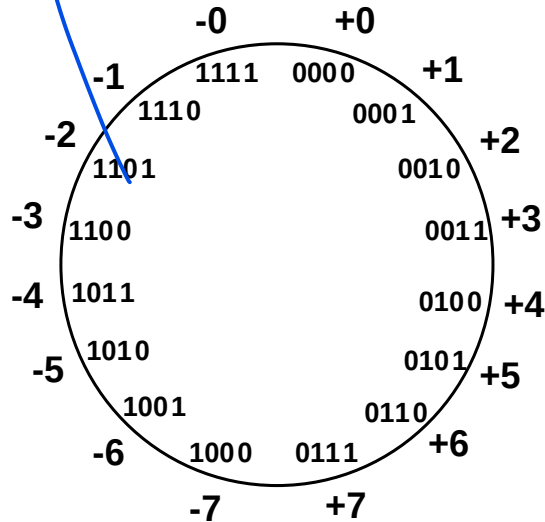


- Bits
- Bytes

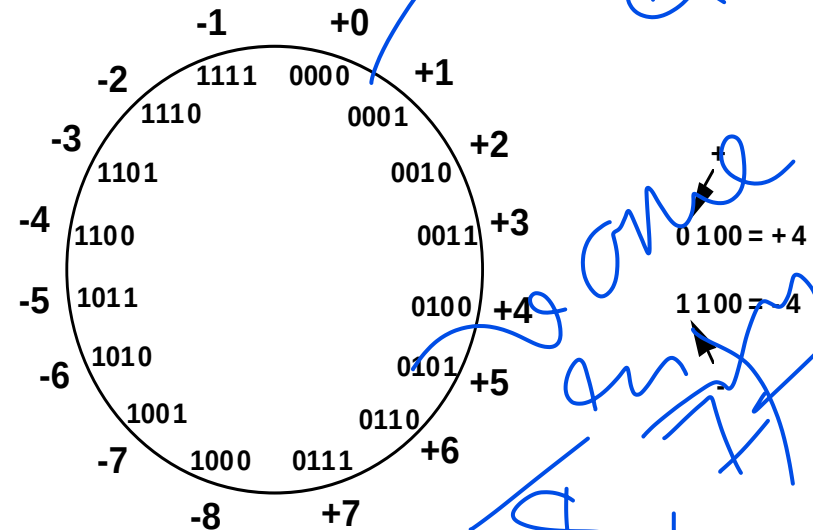
Review of Number Representation



SM Form



1's C



2's C

Positive Integers

- Decimal system: made of 10 digits, $\{0,1,2, \dots, 9\}$

$$41 = 4 \times 10^1 + 1 \times 10^0$$

$$255 = 2 \times 10^2 + 5 \times 10^1 + 5 \times 10^0$$

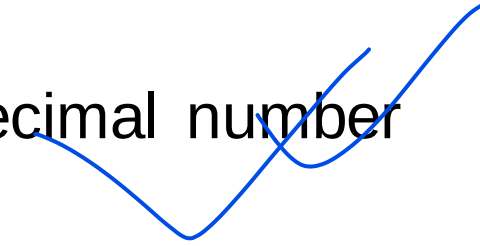
- Binary system: made of two digits, $\{0,1\}$

$$\begin{aligned} 00101001 &= 0 \times 2^7 + 0 \times 2^6 + 1 \times 2^5 + 0 \times 2^4 \\ &\quad + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 \\ &= 32 + 8 + 1 = 41 \end{aligned}$$

- Hexadecimal System: the digits in base 16 are 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, and F

0x as a prefix is the convention for hexadecimal number

Ex: 0x4B2A



Hexadecimal System

Denary/Decimal	Binary	Hexadecimal
Base 10 Number System	Base 2 Number System	Base 16 Number System
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

$$0x428 = ?$$

$$= 4 \times 16^2 + 2 \times 16^1 + 8 \times 16^0$$

$$= 1024 + 32 + 8 = 1064$$

$$0xAD4 = ?$$

$$= 10 \times 16^2 + 13 \times 16^1 + 4 \times 16^0$$

$$= 2560 + 208 + 4 = 2772$$

Converting Binary to Hexadecimal

- Mark groups of four (from right)

- Convert each group

10101011

10101011

A B

10101011 is AB in base 16 or 0xAB

1010001101011110

1010001101011110

A 3 5 E

Hence, 1010001101011110 = 0xA35E

Denary/Decimal	Binary	Hexadecimal
Base 10 Number System	Base 2 Number System	Base 16 Number System
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

Integers and Real Numbers

- **Integers: the universe is infinite but discrete**
 - No numbers between consecutive integers, e.g., 5 and 6
 - A countable (finite) number of items in a finite range
 - Referred to as fixed-point numbers
- **Real numbers – the universe is infinite and continuous**
 - Fractions represented by decimal notation
 - Rational numbers, e.g., $5/2 = 2.5$
 - Irrational numbers, e.g., $22/7 = 3.14159265 \dots$
 - Infinite numbers exist even in the smallest range
 - Referred to as floating-point numbers

Wide Range of Numbers

$$9.76 \times 10^{14}$$

- A large number:

$$976,000,000,000,000 = 9.76 \times 10^{14}$$

- A small number:

$$0.00000000000000976 = 9.76 \times 10^{-14}$$

$$9.76 \times 10^{-14}$$

Scientific Notation

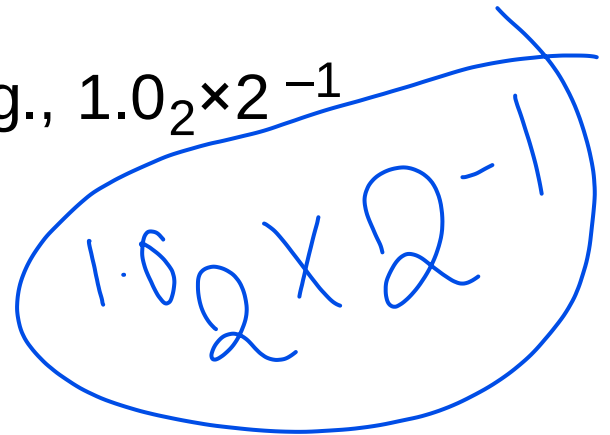
- Decimal numbers

- 0.513×10^5 , 5.13×10^4 and 51.3×10^3 are written in scientific notation.

- 5.13×10^4 is the normalized scientific notation.

- Binary numbers

- Base 2
- Binary point – multiplication by 2 moves the point to the right.
- Normalized scientific notation, e.g., $1.0_2 \times 2^{-1}$



A handwritten example of normalized binary scientific notation, $1.0_2 \times 2^{-1}$, enclosed in a blue oval. The text is written in blue ink.

Floating Point Numbers

- General format

$$\pm 1.bbbbb_2 \times 2^{eeee}$$

$$\pm 1.bbbbb_2 \times 2^{eeee}$$

or

$$(-1)^S \times (1+F) \times 2^E$$

- Where

- S = sign, 0 for positive, 1 for negative
- F = fraction (or mantissa) as a binary integer, 1+F is called significand
- E = exponent as a binary integer, positive or negative (two's complement ?? / SM ??)

Binary to Decimal Conversion

Binary $(-1)^S (1.b_1b_2b_3b_4) \times 2^E$

Decimal $(-1)^S \times (1 + b_1 \times 2^{-1} + b_2 \times 2^{-2} + b_3 \times 2^{-3} + b_4 \times 2^{-4}) \times 2^E$

Example: -1.1100×2^{-2} (binary)

$$= - (1 + 2^{-1} + 2^{-2}) \times 2^{-2}$$

$$= - (1 + 0.5 + 0.25) / 4$$

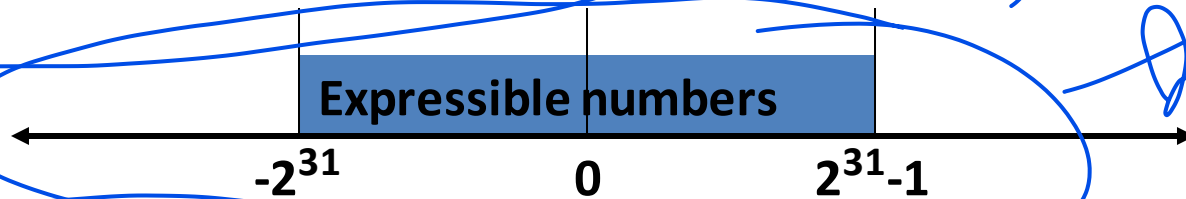
$$= - 1.75 / 4$$

$$= - 0.4375 \text{ (decimal)}$$

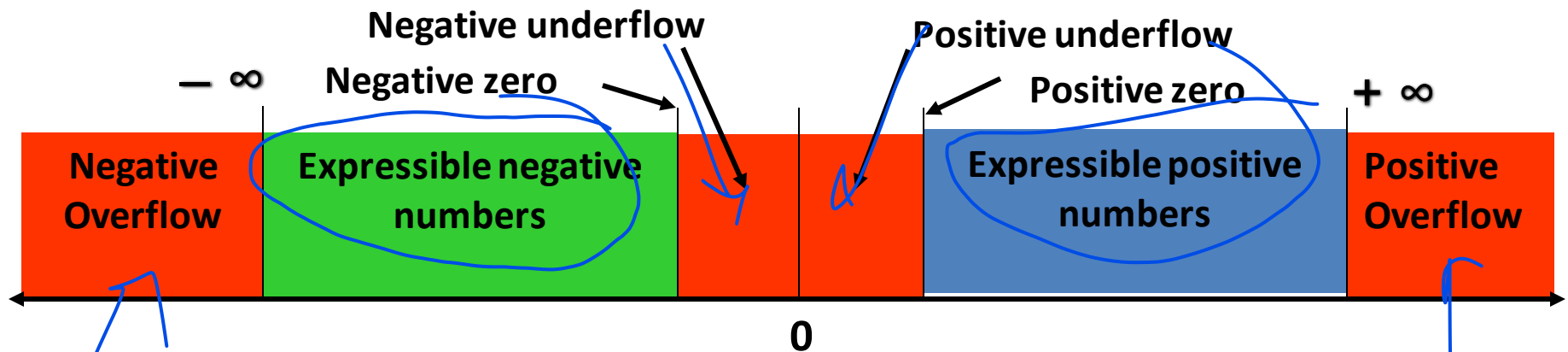
0.4375

Numbers in 32-bit Formats

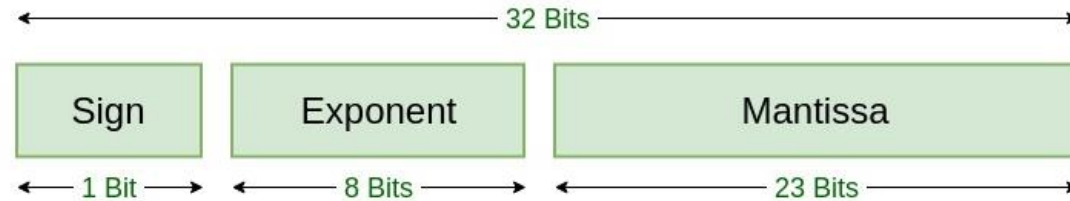
- Two's complement integers



- Floating point numbers

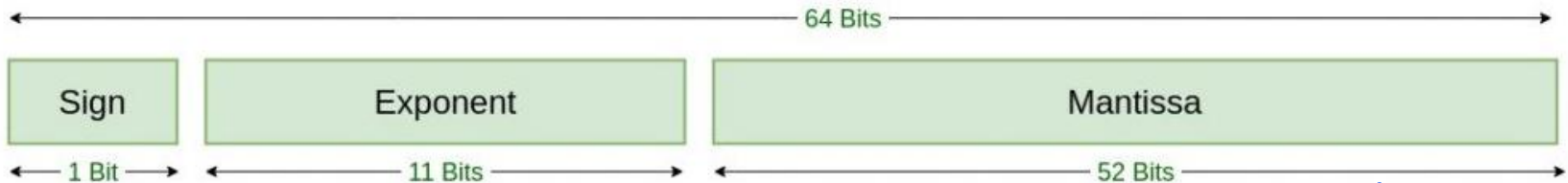


IEEE 754 Floating Point Standards



Single Precision
IEEE 754 Floating-Point Standard

Handwritten notes: 1 bit-
exponent
23

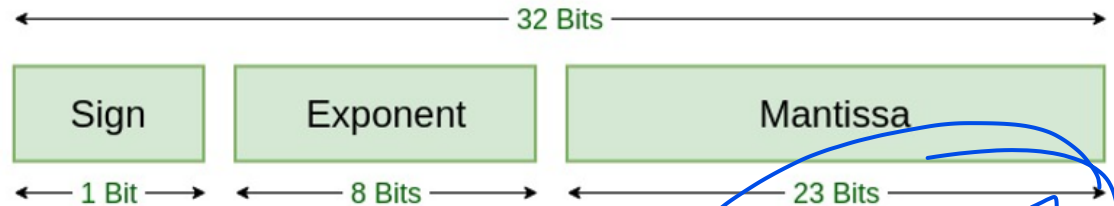


Double Precision
IEEE 754 Floating-Point Standard

Handwritten notes: 1 B.1
4 Bits
52

IEEE 754 Floating Point Standard

- Single Precision Floating point numbers

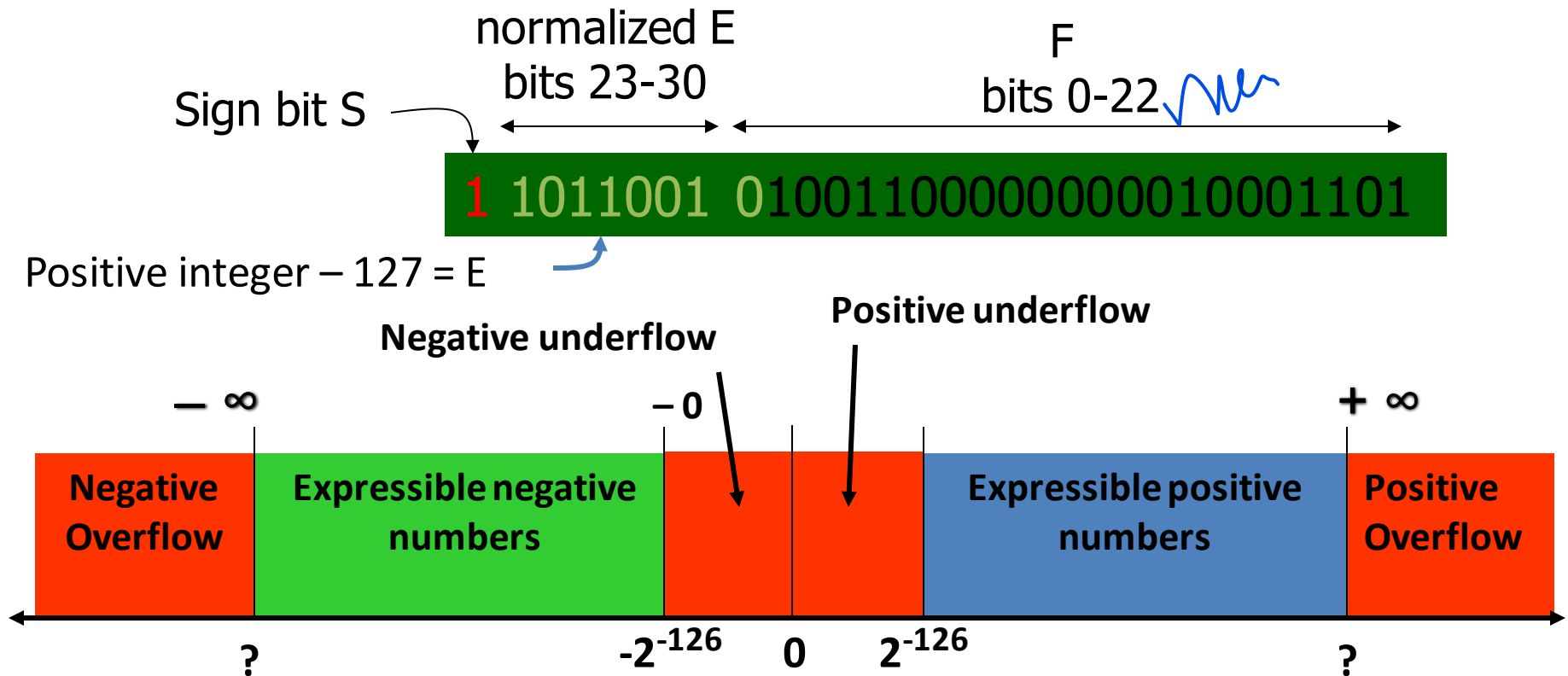


- Biased exponent: true exponent range
 - $[-126, 127]$ is changed to $[1, 254]$
 - Biased exponent is an 8-bit positive binary integer.
 - True exponent obtained by subtracting 127ten or 01111111two
- First bit of significand is always 1
 - $\pm 1.bbbb \dots b \times 2^E$
 - 1 before the binary point is implicitly assumed.
 - Significand field represents 23 bit fraction after the binary point.

Handwritten note: 128, 127

IEEE 754 Floating Point Standard

- Single Precision Floating point numbers



0 1010001

Examples

Ex/20

Biased exponent (1-254), bias 127 (01111111) to be subtracted



$$1.1010001 \times 2^{10100} = \text{0 } 10010011 \text{ } 101000100000000000000000 = 1.6328125 \times 2^{20}$$

$$-1.1010001 \times 2^{10100} = \text{1 } 10010011 \text{ } 101000100000000000000000 = -1.6328125 \times 2^{20}$$

$$1.1010001 \times 2^{-10100} = \text{0 } 01101011 \text{ } 101000100000000000000000 = 1.6328125 \times 2^{-20}$$

$$-1.1010001 \times 2^{-10100} = \text{1 } 01101011 \text{ } 101000100000000000000000 = -1.6328125 \times 2^{-20}$$



Conversion From Hex to Decimal

- R1= 0x42200000

$$0 \text{ } 100 \text{ } 0010 \text{ } 0010 \text{ } 0000 \text{ } 0000 \text{ } 0000 \text{ } 0000 \text{ } 0000 \text{ } 0000 = + 1.010 \times 2^5$$

$$101000 = +40$$

- R2=0xC1200000

$$1 \text{ } 100 \text{ } 0001 \text{ } 0010 \text{ } 0000 \text{ } 0000 \text{ } 0000 \text{ } 0000 \text{ } 0000 \text{ } 0000 = - 1.01 \times 2^3$$

$$1010 = -10$$

- R3=0xC0800000

$$1 \text{ } 100 \text{ } 0000 \text{ } 1000 \text{ } 0000 \text{ } 0000 \text{ } 0000 \text{ } 0000 \text{ } 0000 \text{ } 0000 = - 1.00 \times 2^2$$

$$0100 = -4$$

0 00000000 00000000000000000000000000000000

Fraction

-
- The diagram illustrates the IEEE 754 floating-point standard number line, showing the distribution of floating-point numbers across different ranges. The number line is marked with $-\infty$, Negative zero, 0, Positive zero, and $+\infty$.
- The regions are defined as follows:
- Negative Overflow:** The region to the left of $-\infty$, colored red.
 - Expressible negative numbers:** The region between $-\infty$ and Negative zero, colored green.
 - Negative underflow:** The region between Negative zero and 0, colored red. An arrow points to this region from the label "Negative underflow".
 - Positive underflow:** The region between 0 and Positive zero, colored red. An arrow points to this region from the label "Positive underflow".
 - Expressible positive numbers:** The region between Positive zero and $+\infty$, colored blue.
 - Positive Overflow:** The region to the right of $+\infty$, colored red.

1 00000000 000000000000000000000000

Biased exponent Fraction

-
- The diagram illustrates the IEEE 754 floating-point standard number line, showing the distribution of floating-point numbers across different ranges. The number line is marked with $-\infty$, Negative zero, 0, Positive zero, and $+\infty$.
- The regions are defined as follows:
- Negative Overflow:** The region to the left of $-\infty$, colored red.
 - Expressible negative numbers:** The region between $-\infty$ and Negative zero, colored green.
 - Negative underflow:** The region between Negative zero and 0, colored red. An arrow points to this region from the label "Negative underflow".
 - Positive underflow:** The region between 0 and Positive zero, colored red. An arrow points to this region from the label "Positive underflow".
 - Expressible positive numbers:** The region between Positive zero and $+\infty$, colored blue.
 - Positive Overflow:** The region to the right of $+\infty$, colored red.

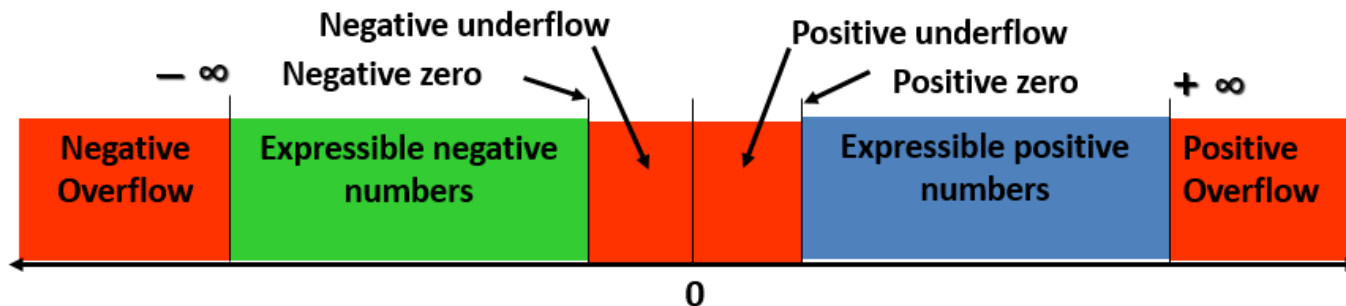
Positive Infinity in IEEE 754

0 11111111 00000000000000000000000000000000

Biased
exponent

Fraction

- $+ 1.0 \times 2^{128}$
- Greater than the largest positive number in single-precision IEEE 754 standard.
- Interpreted as $+\infty$
- If true exponent > 127 , then the number is greater than ∞ . It is called “not a number” or NaN and may be interpreted as ∞ .



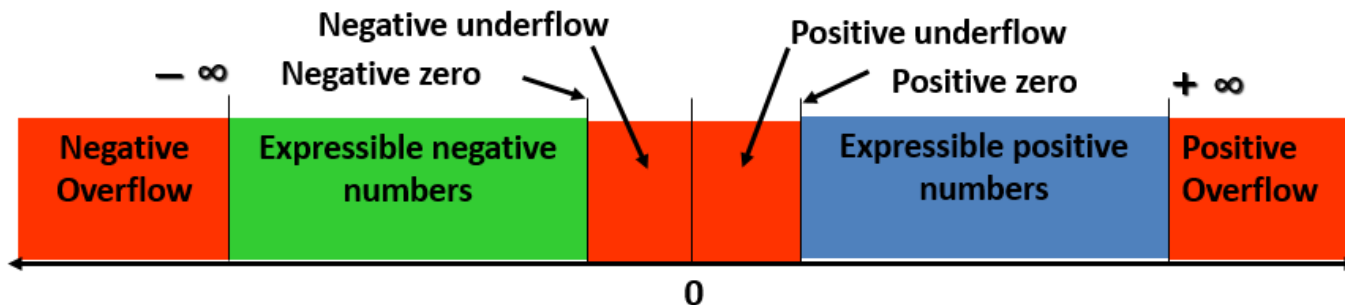
Negative Infinity in IEEE 754

1 11111111 000000000000000000000000

Biased
exponent

Fraction

- -1.0×2^{128}
- Smaller than the smallest negative number in single-precision IEEE 754 standard.
- Interpreted as $-\infty$
- If true exponent > 127 , then the number is less than $-\infty$. It is called “not a number” or NaN and may be interpreted as $-\infty$.



FP Addition and Subtraction

1. **Significand alignment:** Right shift significand of smaller exponent until two exponents match.
2. **Addition:** Add significands and report error if overflow occurs. If significand = 0, return result as 0.
3. **Normalization**
 - Shift significand bits to normalize.
 - report overflow or underflow if exponent goes out of range.
4. **Rounding**

Example (4 Significant Fraction Bits)

- Subtraction: $0.5_{\text{ten}} - 0.4375_{\text{ten}}$

- Floating point numbers to be added

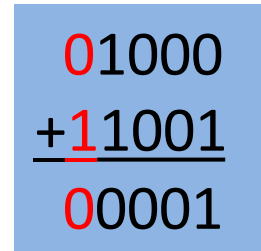
$$1.000_{\text{two}} \times 2^{-1} \text{ and } -1.110_{\text{two}} \times 2^{-2}$$

- Significand of lesser exponent is shifted right until exponents match

$$-1.110_{\text{two}} \times 2^{-2} \rightarrow -0.111_{\text{two}} \times 2^{-1}$$

- Add significands, $1.000_{\text{two}} + (-0.111_{\text{two}})$

$$\text{Result is } 0.001_{\text{two}} \times 2^{-1}$$


$$\begin{array}{r} 01000 \\ + \underline{11001} \\ \hline 00001 \end{array}$$

2's complement addition,
one bit added for sign

- Normalize, $1.000_{\text{two}} \times 2^{-4}$

No overflow/underflow since

$$127 \geq \text{exponent} \geq -126$$

$$1.000_{\text{two}} \times 2^{-4} = (1+0)/16 = 0.0625_{\text{ten}}$$

Thanks